

Chapter 9

2D physics

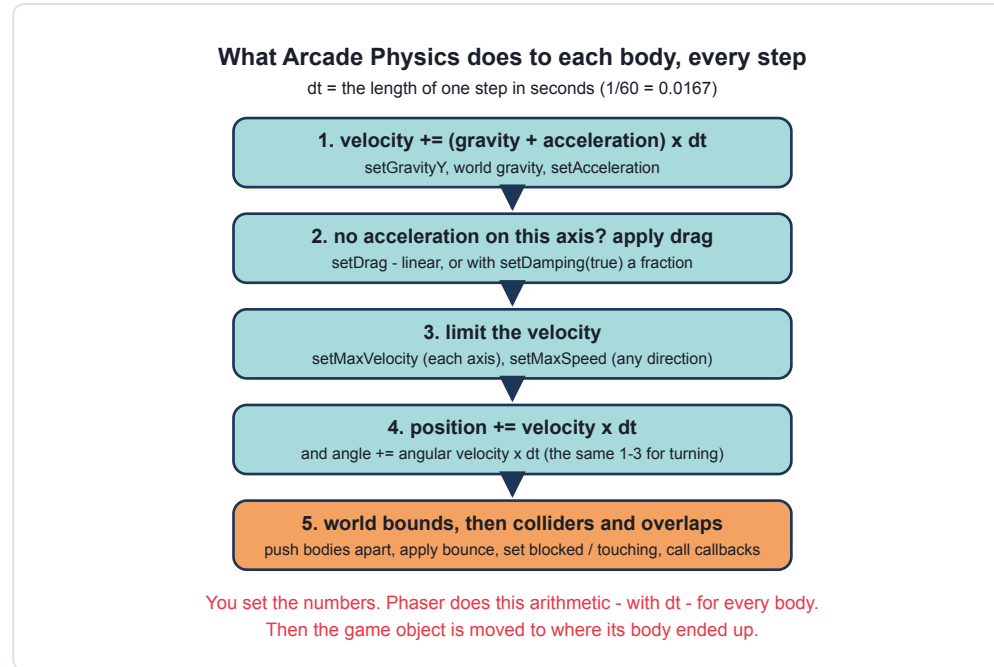
Velocity, gravity, drag, bounce - and a spaceship



Today

- what the physics engine does to every body, every step
- gravity, bounce, drag and damping, mass, friction
- kinds of body: dynamic, immovable, static
- a trap: groups reset their bodies
- turning, thrust, top speed, wrapping round the world
- Matter.js: shapes that rotate and stack

Every body, every step



You set the numbers. Phaser does the `delta` maths.

Velocity and acceleration

- **velocity** - pixels per second. Changes the **position**
- **acceleration** - pixels per second, per second. Changes the **velocity**
- **gravity** - an acceleration every body shares

`setVelocityX(200)` - right at 200 px/s, from now on

`setAccelerationX(200)` - 200 px/s faster every second: 600 px/s after three seconds

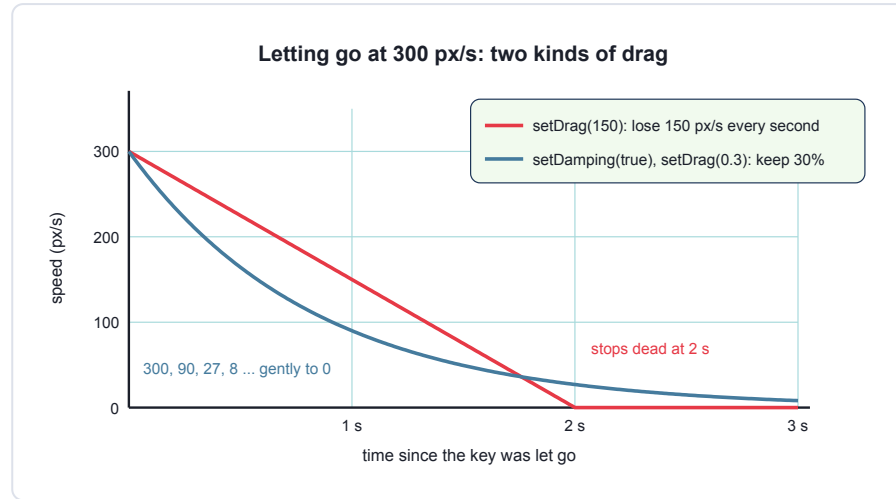
Gravity and bounce

```
// G: the world's gravity - shared by every body (a body can add its own with setGravityY)  
keyboard.on("keydown-G", () => {  
  this.gravityIndex = (this.gravityIndex + 1) % GRAVITY_CHOICES.length;  
  this.physics.world.gravity.y = GRAVITY_CHOICES[this.gravityIndex].value;  
});
```

```
// the physics world starts as big as the game; stop it at the top of the ground instead  
this.physics.world.setBounds(0, 0, 800, FLOOR_Y);
```

- `setBounce(b)` : 0 dead, 1 perfect, more than 1 gains speed
- `setCollideWorldBounds(true)` : the world's edges are walls

Drag, two ways



- `setDrag(150)` - lose 150 px/s every second
- `setDamping(true)` + `setDrag(0.3)` - keep 30% every second
- **drag only works when acceleration is 0**

The trap: groups reset their bodies

```
private addBall(x: number, y: number): void {  
    const ball = new Ball(this, x, y);  
    // add to the group FIRST: a physics group resets the bodies it is given to its own defaults  
    // (no bounce, no velocity...), so anything set before this line would be lost  
    this.things.add(ball);  
    bodyOf(ball).onWorldBounds = true;  
    ball.setVelocityX(Phaser.Math.Between(-LAUNCH_SPEED, LAUNCH_SPEED));  
    this.applySettings(ball);  
}
```

No error. No warning. The ball just stops.

Kinds of body

Body	Moves itself?	Pushed?
dynamic	yes	yes, shared by mass
<code>setPushable(false)</code>	yes	no - pushes back
<code>setImmovable(true)</code>	if given a velocity	never
static	never	never (cheapest)

Arcade **friction** belongs to a moving immovable body: how much of its movement it passes to a rider. `setFriction(1, 0)` carries crates; `0` slides out from under them.

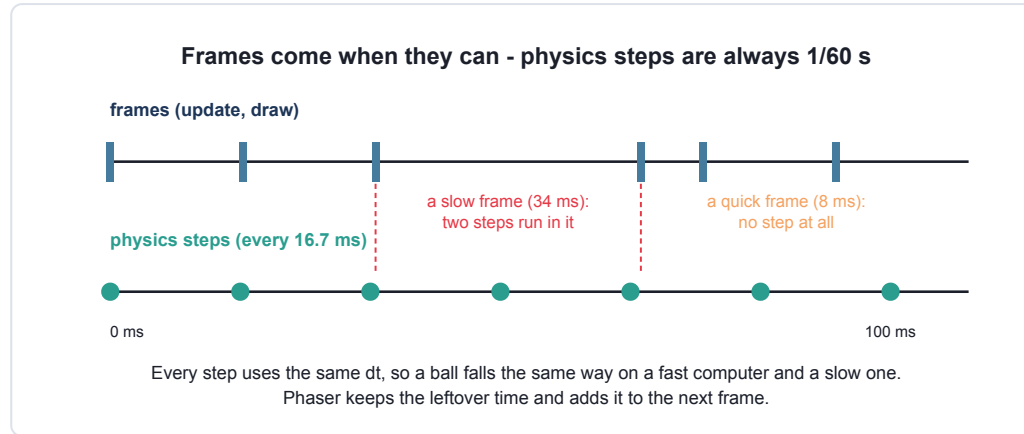
What is it touching?

- `body.blocked.down` - resting on something that cannot give way
- `body.touching.down` - touching any body below
- worked out afresh every step: check in `update()`

```
this.physics.world.on(  
  Phaser.Physics.Arcade.Events.WORLD_BOUNDS,  
  (body: Phaser.Physics.Arcade.Body, _up: boolean, down: boolean) => {  
    // ...  
  },  
);
```

Only for bodies with `onWorldBounds = true`

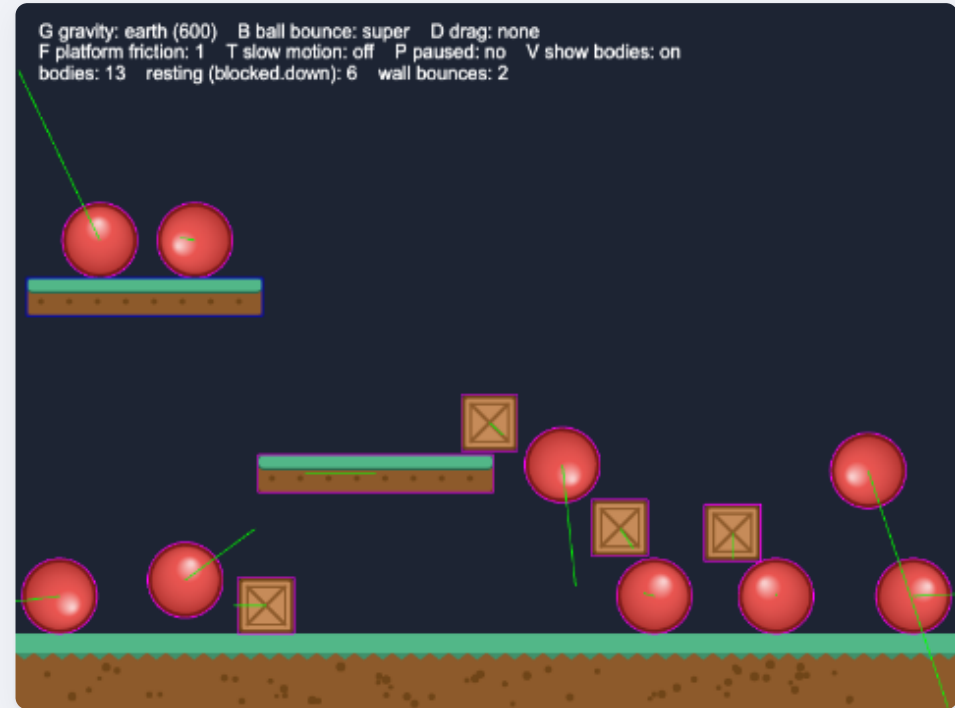
The physics clock



- `this.physics.pause()` / `resume()` - the scene keeps running
- `world.timeScale = 3` - slow motion (**bigger is slower**)

Try it: the playground

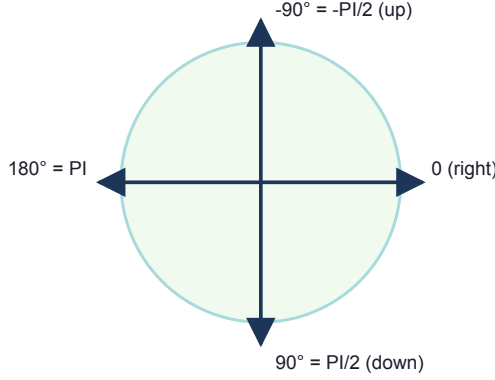
- G B D F T P - change the laws of physics
- V - see the real bodies
- watch a ball roll: the picture turns, the **body never does**
- drop a crate on the platform; press F



Which way is angle 0?

Which way is angle 0?

Phaser's angles (and the maths)



180° = π

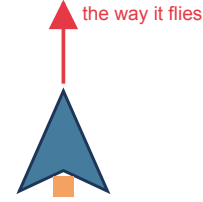
0 (right)

-90° = $-\pi/2$ (up)

90° = $\pi/2$ (down)

y goes DOWN the screen, so angles go clockwise

The ship's picture points up



the way it flies

drawn like this, its rotation is 0
- but it faces $-\pi/2$ (up)

facing = rotation - $\text{Math.PI} / 2$

Turning and thrust

```
public turn(direction: number): void {
    this.setAngularAcceleration(direction * TURN_ACCELERATION);
}

public thrust(on: boolean): void {
    if (on) {
        ...
        const facing = this.rotation - Math.PI / 2;
        ...
        this.scene.physics.velocityFromRotation(facing, THRUST, this.arcadeBody.acceleration);
    } else {
        ...
        this.setAcceleration(0);
    }
}
```

Drift, top speed, wrap

```
this.setDamping(true);  
this.setDrag(DAMPING);  
...  
this.arcadeBody.setMaxSpeed(MAX_SPEED);
```

DAMPING = 0.6 - keep 60% a second. setMaxSpeed - in any direction

- setMaxVelocity(x, y) limits each axis: diagonals go 1.4 times faster

```
this.physics.world.wrap(this.ship, SHIP_WRAP);  
this.physics.world.wrap(this.rocks, ROCK_WRAP);
```

Try it: the space ship

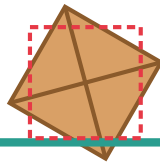
- LEFT / RIGHT turn, UP thrusts, SPACE fires
- turn round and thrust to stop
- bullets get the ship's velocity added
- `this.physics.pause()` at game over



Matter.js: why a second engine?

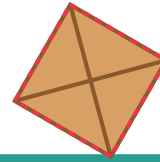
The same tilted crate, in each engine

Arcade Physics



the picture turns - the body (red) does not:
always an upright box, or a circle
it cannot tip over, and its corners poke through

Matter.js



the body turns with the picture: any angle,
any convex shape - so it rests on its corner
and topples over, as a real crate would

```
physics: {  
  default: "matter",  
  matter: {  
    ...  
    gravity: { x: 0, y: 1 },  
  },  
}
```

Matter game objects

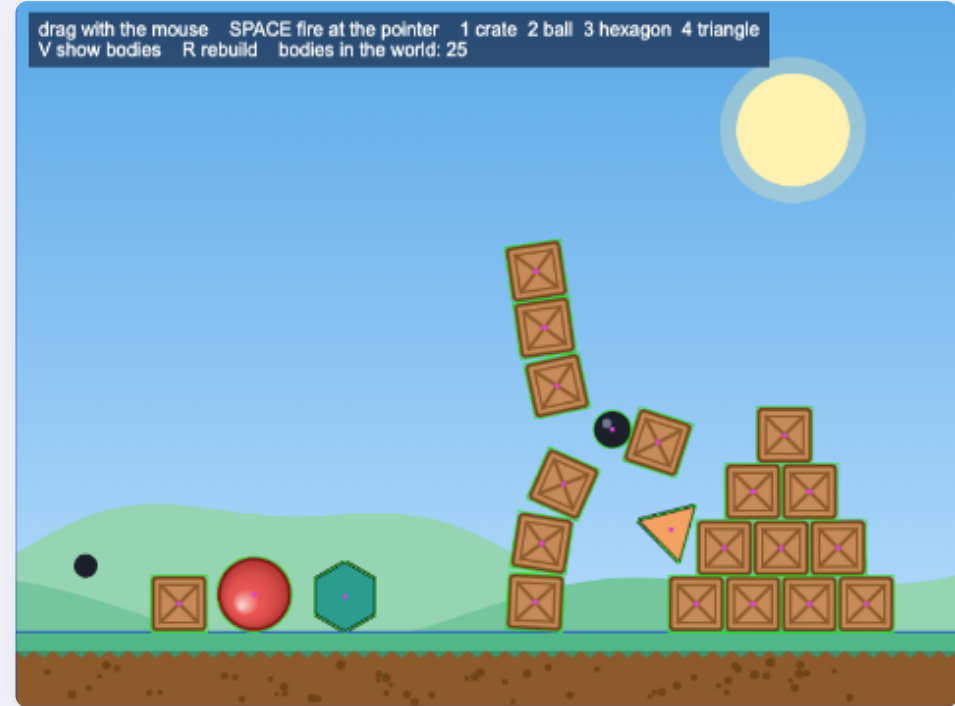
```
export class Crate extends Phaser.Physics.Matter.Image {  
  constructor(scene: Phaser.Scene, x: number, y: number) {  
    super(scene.matter.world, x, y, CRATE_KEY, undefined, {  
      friction: 0.6,           // wood on wood: grippy, so a stack stays up (0 = ice, 1 = rubber)  
      restitution: 0.05,       // bounce: crates hardly bounce at all  
    });  
    scene.add.existing(this);  
  }  
}
```

- `shape: { type: "circle", radius }, { type: "polygon", sides, radius }`
- `density` gives mass; velocities are **pixels per step**

Try it: knock it down

```
this.matter.add.mouseSpring({ stiffness: 0.2 });
```

- drag anything; SPACE fires at the pointer
- 1 - 4 drop shapes; V shows them
- Arcade for most games; Matter when the game *is* the physics



Summary

- set velocity and acceleration - the engine integrates, in fixed steps
- gravity, bounce, drag (linear or damping), max speed, mass, friction
- dynamic, immovable, static; `blocked` , `touching` , `"worldbounds"`
- groups reset bodies: **add first, then set up**
- angular velocity and acceleration; `velocityFromRotation` ; `world.wrap`
- Matter: real shapes, rotation, stacking - in its own units

Challenges

1. **Gravity switch** - arrow keys choose which way gravity pulls
2. **Bouncy and dead** - a light super-ball and a heavy dead one
3. **Brakes and reverse** - DOWN for retro-rockets, SHIFT to brake
4. **Rocks that split** - big to medium to small
5. **Demolition** - five cannonballs, count the crates down
6. **Pinball flipper** - a pinned Matter body, flicked with Z

Next: **Chapter 10 - Tilemaps**